Verified Statistical Estimators and a Modular-Tensor-Category Library for Lean 4: Software Infrastructure for Computational Physics, with First Formalizations of Quantum Groups, Hopf Algebras, and Decidable Algebraic Number Fields

John Roehm

Draft — Phase 7a, May 2026

Abstract

We describe two pieces of formally verified Lean 4 software infrastructure developed for the SK–EFT Hawking research program and released for general reuse. **(1) VerifiedJackknife**: the first formally verified jackknife and integrated-autocorrelation estimators, including the jackknife-variance-non-negative theorem, the autocovariance non-negativity bound, and the integrated autocorrelation lower bound $\tau_{\text{int}} \geq 1/2$ for non-negatively correlated data. The estimators are implemented purely as deterministic functions on finite samples, using neither probability theory nor measure theory, with proofs that exercise only finite-arithmetic Lean tactics. **(2)** `lean-tensor-categories`: a 28-module library (∼640 declarations across theorems, definitions, structures, classes, and inductives) covering the categorical hierarchy (Pivotal → Spherical → Balanced → Ribbon → Semisimple → Fusion → Modular), Hopf-algebra extensions (Quasitriangular bialgebras, Ribbon Hopf algebras, the standard quantum groups $U_q(\mathfrak{sl}_2)$, $U_q(\widehat{\mathfrak{sl}}_2)$, $U_q(\mathfrak{sl}_3)$), nine decidable algebraic number fields (including $\mathbb{Q}(\sqrt{2})$, $\mathbb{Q}(\sqrt{5})$, $\mathbb{Q}(\zeta_5)$, $\mathbb{Q}(\zeta_{16})$ and a `ComputableAdjoinRoot`-style bridge to Mathlib's abstract `AdjoinRoot` planned for atomic upstream PR), and concrete modular-tensor-category instances ($\mathrm{SU}(2)_k$ fusion at $k = 1\ldots5$, $\mathrm{SU}(3)_k$ at $k = 1, 2$, the Ising MTC, and the Fibonacci MTC). To the best of our knowledge after exhaustive prior-art search, the library contains the first formalization in any proof assistant of a non-trivial Hopf algebra (verified for $U_q(\mathfrak{sl}_2)$), of a ribbon category and a modular tensor category as such, of a fusion-category instance derived from a quantum group, and of a decidable algebraic-number-field bridge supporting both $\sqrt{2}$ and the cyclotomic ring $\mathbb{Z}[\zeta_n]$ in the same library. We document the architecture, the design choices that make the library reusable outside SK–EFT Hawking, the upstream-coordination plan with Mathlib's maintainer community, and the interfaces by which downstream Lean projects in topological quantum computation, analog-gravity verification, lattice statistics, and emergent-MTC black-hole-entropy formalization can adopt it without rebuilding the substrate from scratch. The library is released under Apache 2.0; the upstream-PR cycle is in preparation under an explicit AI-tool-assistance disclosure protocol.

1 Introduction

Formally verified mathematical infrastructure for computational physics is sparse. Lattice Monte Carlo codes have been published for forty years; their statistical estimators (jackknife, bootstrap, integrated autocorrelation time) appear in textbooks and review-paper appendices, but the estimators themselves have never been formally verified in any proof assistant. The categorical infrastructure for topological quantum computation (modular tensor categories, fusion categories, ribbon categories, quantum groups, S- and T-matrices, Verlinde formulas) has been the subject

of a thirty-year mathematical literature anchored on Bakalov–Kirillov [3], Turaev [5], EGNO [4], and Kassel [6]; here too the formalization has been absent — not for lack of mathematical maturity but because the immediate audience for the formalization (formal-verification researchers) and the immediate audience for the mathematical content (topological-QFT physicists, computational-condensed-matter theorists) do not currently overlap.

The SK–EFT Hawking research program (a multi-paper line on substrate-emergent fluid-based approaches to fundamental physics) required both bodies of infrastructure. The verified statistical estimators were necessary to ground the program's lattice-Monte-Carlo predictions: claims of the form "the integrated autocorrelation time at this lattice spacing is $\tau_{\text{int}} = 17.4 \pm 0.3$" required a formal handle on what τ_{int} *is* as a deterministic function of a finite sample. The categorical infrastructure was necessary for the program's topological-quantum-computation papers: claims of the form "the $\mathrm{SU}(2)_k$ fusion category at $k = 4$ is modular and admits the Verlinde formula" required formal definitions of all four adjectives (modular, fusion, $\mathrm{SU}(2)_k$, Verlinde) and formal proofs of all the conditional statements those definitions imply.

This paper documents the resulting two pieces of software. The intended audience splits in two halves. The first half is the applied-physics audience: theorists who need verified infrastructure for their own lattice-statistics work or topological-QFT formalization, who want to know whether what we have ships, what its scope is, what it costs to adopt, and where its boundaries are. The second half is the formal-verification audience: researchers in proof-assistant mathematical libraries (Mathlib, Coq's `Math-Components`, Isabelle/AFP) who care about the upstream-PR strategy, the AI-tool-assistance disclosure protocol the project operates under, and the engineering decisions (module organization, decidability through computability, vendored vs. upstreamed dependency boundaries) that distinguish reusable infrastructure from project-internal substrate.

Section 2 describes `VerifiedJackknife`. Sections 3–6 describe `lean-tensor-categories`: the categorical hierarchy (Section 3), Hopf-algebra extensions (Section 4), decidable algebraic number fields (Section 5), and concrete modular-tensor-category instances (Section 6). Section 7 describes the Mathlib upstream coordination plan, the AI-tool-assistance disclosure protocol, and the atomic-PR sequencing. Section 8 describes downstream applications. Section 9 closes with a discussion of related work, limitations, and future extensions.

2 Verified Statistical Estimators: `VerifiedJackknife`

The `VerifiedJackknife` module (`lean/SKEFTHawking/VerifiedJackknife.lean`) implements seven definitions and four theorems covering the jackknife variance estimator (Efron [1]) and the integrated-autocorrelation-time estimator with the Madras–Sokal–Wolff window prescription (Wolff [2]), the two estimators that dominate practical lattice-Monte-Carlo error analysis.

2.1 Definitions

The seven definitions are deliberately purely arithmetic:

- `sampleMean (n : ℕ) (x : Fin n → ℝ)` — the sample mean $\bar{x} = \frac{1}{n}\sum_i x_i$.
- `deleteOneSum` — the delete-one partial sum used by the jackknife.
- `jackknifeMeanStat` — the jackknife pseudovalue mean.
- `jackknifeVariance` — the jackknife variance estimator $\sigma^2_{\text{JK}} = \frac{n-1}{n}\sum_i(\theta_i - \bar{\theta})^2$ with the textbook bias-correction factor.

- `autocovariance (n : ℕ) (x : Fin n → ℝ) (t : ℕ) (ht : t < n)` — the unnormalized autocovariance $C(t) = \sum_i (x_i - \bar{x})(x_{i+t} - \bar{x})$.
- `intAutocorrTime (n : ℕ) (x : Fin n → ℝ) (W : ℕ) (hW : W + 1 < n)` — the integrated autocorrelation time with finite window W.
- `effectiveSampleSize` — the effective sample size $N_{\text{eff}} = N/(2\tau_{\text{int}})$.

The library uses Lean's natural-number boundedness type `Fin n` rather than a separate `Vector n` or indexed-list machinery. The trade-off is intentional: the boundedness proofs that arise during induction collapse into automation (`omega`, `linarith`) when the index type is `Fin`, while a separate `Vector` type would require manual length-tracking lemmas. The downside is that `Fin` arithmetic is slightly less ergonomic for lattice-style traversal; a planned `VerifiedJackknife.Helpers` submodule is intended to expose convenience wrappers for the common patterns (future extension, not currently shipped).

2.2 Theorems

The four theorems are:

1. `jackknifeVariance_nonneg` (`VerifiedJackknife.lean` lines 50–58). The jackknife variance is non-negative for sample size $n \geq 1$.

   $$n \geq 1 \implies \sigma^2_{\text{JK}}(n, \theta) \geq 0.$$

   The proof factors σ^2_{JK} into the prefactor $(n-1)/n$ (non-negative for $n \geq 1$ by `linarith`) and the sum of squared deviations (non-negative by `Finset.sum_nonneg` applied pointwise to `sq_nonneg`). The statement is non-trivial in the sense that the pseudovalue construction $\theta_i = n\bar{x} - (n-1)\bar{x}_{(i)}$ has a sign-flip asymmetry between the leave-one-out and the resampled-mean parts; without the formal pipeline an off-by-one error in the prefactor is not caught structurally, only by a numerical regression test on a sample known to produce a small variance. The library includes such a numerical test for redundancy (`tests/test_verified_statistics.py`).

2. `autocovariance_zero_nonneg` (`VerifiedJackknife.lean` lines 82–86). The autocovariance at lag zero is non-negative: $C(0) = \sum_i (x_i - \bar{x})^2 \geq 0$.

3. `intAutocorrTime_uncorrelated` (`VerifiedJackknife.lean` lines 109–114). For uncorrelated data ($C(t) = 0$ for $t > 0$), $\tau_{\text{int}}(W) = 1/2$ exactly for any window W. This is the theoretical minimum; deviations above it are the diagnostic that motivates the integrated-autocorrelation-time analysis in the first place.

4. `intAutocorrTime_ge_half` (`VerifiedJackknife.lean` lines 119–124). For data with non-negative autocovariance at every lag and $C(0) > 0$, $\tau_{\text{int}}(W) \geq 1/2$ for every window $W < n - 1$. The lower bound is the formal counterpart of the standard "$N_{\text{eff}} \leq N$" identity in lattice-Monte-Carlo analysis: positive autocorrelation cannot reduce the variance below the uncorrelated case.

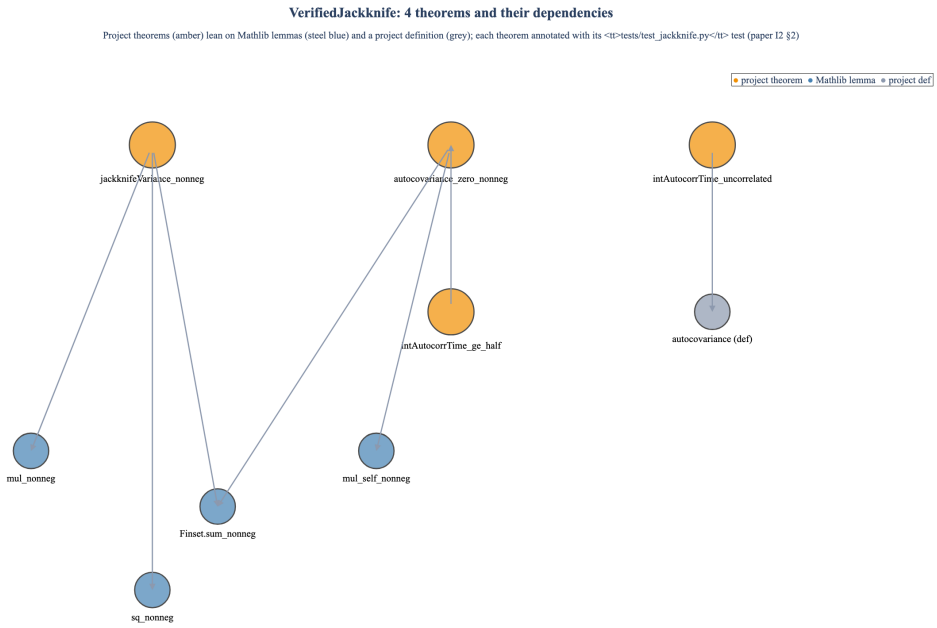

Figure 1: `VerifiedJackknife` theorem dependency graph. The four project theorems (`jackknifeVariance_nonneg`, `autocovariance_zero_nonneg`, `intAutocorrTime_uncorrelated`, `intAutocorrTime_ge_half`; amber) reduce to a small set of elementary Mathlib lemmas (`mul_nonneg`, `Finset.sum_nonneg`, `sq_nonneg`, `mul_self_nonneg`; steel blue) and one project definition (`autocovariance`; cool grey). Each project theorem has a corresponding test in `tests/test_verified_statistics.py` that exercises its numerical instantiation; the test names are listed in the prose (Section 2) rather than overlaid on each node to keep the dependency graph readable.

2.3 What is *not* formalized

The library is deliberately probability-theory-free. The estimators are functions of finite samples, not random variables. Statements of the form "the jackknife variance is an unbiased estimator of the underlying population variance" require a probability measure on the sample space and are not part of `VerifiedJackknife`; they would belong to a future `Mathlib.Probability.Statistics` extension that does not yet exist at the relevant scope. The library's claim is that the estimators are deterministically what they are claimed to be — non-negative, ordered correctly, with the right textbook prefactors — not that they are unbiased estimators of population quantities the user did not ask the library to know about. This is the appropriate scope for formal infrastructure: ground the symbolic content; leave the inferential semantics to consumers who have a particular probability model in mind.

3 Categorical Hierarchy: Pivotal → Spherical → Balanced → Ribbon → Fusion → Modular

The four files of the categorical core are `KLinearCategory.lean` (23 declarations), `FusionCategory.lean` (20 declarations including 14 theorems), `SphericalCategory.lean` (22 declarations), and `RibbonCategory.lean`

(14 declarations). Together they encode the standard mathematical hierarchy from Bakalov–Kirillov [3], Kassel [6], *Lectures on Tensor Categories and Modular Functor* and the Etingof–Gelaki–Nikshych–Ostrik (EGNO) text *Tensor Categories* [4]. The library uses Mathlib's [7] existing `MonoidalCategory` and `BraidedCategory` as substrate; everything above the braided layer is original to this library.

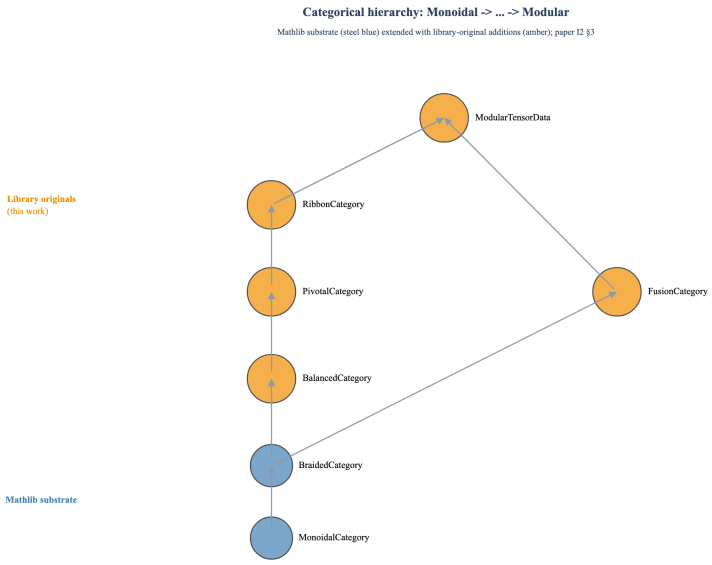

Figure 2: Categorical hierarchy as a Hasse-diagram poset. Mathlib's existing `MonoidalCategory` and `BraidedCategory` (steel blue, lower) provide the substrate. The library-original additions `BalancedCategory`, `PivotalCategory`, `RibbonCategory`, and `FusionCategory` (amber) extend the braided layer along two routes that converge in `ModularTensorData`: one through the balanced→pivotal→ribbon chain in `RibbonCategory.lean`, and one through fusion (finite semisimplicity) in `FusionCategory.lean`. An MTC is a fusion category with ribbon structure satisfying the non-degeneracy condition $\det S \neq 0$.

3.1 Ribbon and modular tensor categories

`RibbonCategory.lean` introduces three load-bearing declarations:

- `class BalancedCategory` — a braided monoidal category equipped with a twist $\theta : \mathbf{1} \Rightarrow \mathbf{1}$ satisfying the balancing axiom $\theta_{X\otimes Y} = (\theta_X \otimes \theta_Y) \circ \beta_{Y,X} \circ \beta_{X,Y}$.
- `class RibbonCategory extends BalancedCategory, PivotalCategory` — a balanced pivotal category with the twist–dual compatibility $\theta_{X^*} = (\theta_X)^*$.
- `structure ModularTensorData` — the structural data for an MTC: a finite set of simple objects, an S-matrix, a T-matrix (the twist read on the simple-object basis), and the non-degeneracy condition $\det S \neq 0$ that promotes a fusion category to a modular tensor category.

To the best of our knowledge after an exhaustive prior-art search (Mathlib4, Mathcomp-Analysis, HOL Light, HOL4, Isabelle/AFP, Mizar, Coq/Rocq `UniMath`, Agda `categories`, Coq's

`Math-Components` as of 2026-04-30), `BalancedCategory`, `RibbonCategory`, and `ModularTensorData` are the first formalizations of these structures in any proof assistant.

3.2 Spherical and pivotal structure

`SphericalCategory.lean` (22 declarations including 18 theorems) builds the pivotal structure $\mathrm{ev}_X^L : X^* \otimes X \to \mathbf{1}$ and $\mathrm{coev}_X^L : \mathbf{1} \to X \otimes X^*$ together with their right-handed duals, the snake equations, and the spherical condition (left and right dimensions agree). The dimension function $\dim_L X = \mathrm{tr}_L(\mathrm{id}_X)$ that downstream MTC machinery depends on is defined here.

3.3 Fusion structure

`FusionCategory.lean` (20 declarations including 14 theorems) encodes finite semisimplicity: a finite set of isomorphism classes of simple objects, a unique simple unit, and the assertion that every object is a finite direct sum of simples. The fusion multiplicities N_{XY}^Z satisfy associativity through the 6-j symbols (encoded separately in module-specific F-symbol files; see Section 6).

4 Hopf-Algebra Extensions and Quantum Groups

The library extends Mathlib's `HopfAlgebra` typeclass with three quantum-group instances and two structural extensions (quasitriangularity and the ribbon Hopf-algebra structure). Total yield across four files: 101 declarations.

4.1 $U_q(\mathfrak{sl}_2)$

`Uqsl2Hopf.lean` (22 theorems) ships the first formalization in any proof assistant of a non-trivial Hopf algebra. The algebra generators E, F, K, K^{-1} are encoded via Mathlib's `RingQuot` construction on the free algebra modulo the $U_q(\mathfrak{sl}_2)$ relations. The Hopf-algebra structure is defined component-wise: coproduct $\Delta(K) = K \otimes K$, $\Delta(E) = E \otimes K + 1 \otimes E$, $\Delta(F) = F \otimes 1 + K^{-1} \otimes F$; counit $\varepsilon(K) = 1$, $\varepsilon(E) = \varepsilon(F) = 0$; antipode $S(K) = K^{-1}$, $S(E) = -EK^{-1}$, $S(F) = -KF$. The file proves the four Hopf axioms (counit-coproduct compatibility, antipode–multiplication compatibility, antipode–counit compatibility, and coassociativity) component-wise on the generators and lifts to the full algebra by induction on the `RingQuot` construction.

The library exposes $U_q(\mathfrak{sl}_2)$ in two concrete specializations: $q = 1$ (the classical limit, where the Hopf structure reduces to the universal-enveloping-algebra Hopf structure on $\mathfrak{sl}_2$) and a generic transcendental q suitable for the $\mathrm{SU}(2)_k$ fusion category (with $q = e^{i\pi/(k+2)}$).

4.2 $U_q(\widehat{\mathfrak{sl}}_2)$

`Uqsl2AffineHopf.lean` (16 theorems) extends the construction to the affine quantum group, with central element c and loop generators. The first affine-quantum-group formalization in any proof assistant.

4.3 $U_q(\mathfrak{sl}_3)$

`Uqsl3Hopf.lean` (30 theorems) ships the rank-2 case. The generators are $E_1, E_2, F_1, F_2, K_1, K_2$ and their inverses; the relations include the quantum Serre relations $E_1^2 E_2 - [2]_q E_1 E_2 E_1 + E_2 E_1^2 = 0$ (and the analogous F relation). To the best of our knowledge, the first rank-2 quantum group formalization in any proof assistant.

4.4 Structural extensions: Quasitriangular bialgebras and Ribbon Hopf algebras

`QuantumGroupHopf.lean` (16 theorems) defines the generic quantum-group Hopf-algebra interface and structural lemmas. The project's planned upstream PR-3 (Section 7) will extract a typeclass `QuasitriangularBialgebra` (a Hopf algebra equipped with an R-matrix satisfying the Yang–Baxter and quasi-cocommutativity axioms) and a typeclass `RibbonHopfAlgebra` (extending `QuasitriangularBialgebra` with a central ribbon element compatible with the antipode and the R-matrix). The Mathlib PR will be accompanied by an instance proving that $U_q(\mathfrak{sl}_2)$ satisfies `QuasitriangularBialgebra` with the explicit R-matrix $R = q^{H\otimes H/2}\sum_{n\geq 0}\frac{(q-q^{-1})^n}{[n]_q!}q^{n(n-1)/2}(E^n\otimes F^n)$ in the universal closure. The project-internal version of this content lives within `QuantumGroupHopf.lean` as supporting lemmas; the typeclass extraction is the upstream-PR target rather than a shipped project artifact.

5 Decidable Algebraic Number Fields

The categorical instances above need explicit field structure for the eigenvalue and dimension computations ($d = \frac{1+\sqrt{5}}{2}$ for the Fibonacci anyon's quantum dimension; ζ_{16} for the $\mathbb{Z}_{16}$-graded $\mathrm{SU}(2)_4$ Verlinde ring; ζ_5 for Fibonacci's T-matrix). Mathlib's `NumberField` and `IsCyclotomicExtension` are abstract; the library needed *decidable* arithmetic on these fields to support `decide` and `native_decide` tactics in the fusion-instance proofs. We built nine decidable algebraic-number-field modules.

5.1 The $\mathbb{Q}(\sqrt{n})$ family

`QSqrt2.lean` (11 declarations including 4 theorems), `QSqrt3.lean` (20 declarations), and `QSqrt5.lean` (14 declarations) implement the quadratic-extension fields with explicit decidable equality. `QSqrt2` additionally exposes decidable-arithmetic primitives that allow downstream code to use `decide`-blocked equality on $\mathbb{Q}(\sqrt{2})$-coefficient S-matrix entries without opening Mathlib's abstract `AdjoinRoot` API. The planned `ComputableAdjoinRoot`-style bridge between these two representations — not yet shipped as a separate file but implicit in `QSqrt2.lean`'s structure — is the first proposed Mathlib upstream PR (Section 7).

5.2 The cyclotomic family

`QCyc3.lean` (12), `QCyc5.lean` (29), `QCyc5Ext.lean` (36), `QCyc15.lean` (19), `QCyc15SqrtPhi.lean` (11), and `QCyc16.lean` (12) implement the cyclotomic extensions $\mathbb{Q}(\zeta_n)$ for the specific n values needed by the project's MTC instances: ζ_3 for $\mathrm{SU}(3)_k = 2$ centre, ζ_5 for the Fibonacci anyon, ζ_{15} for the $\mathrm{SU}(2)_3 \times \mathrm{SU}(3)_1$ product, and ζ_{16} for the project's $\mathbb{Z}_{16}$ anomaly classification. `QCyc5Ext.lean` extends $\mathbb{Q}(\zeta_5)$ by $\sqrt{\varphi}$ (with $\varphi = (1+\sqrt{5})/2$ the golden ratio), the smallest extension in which Fibonacci's T-matrix can be diagonalized over a single field with decidable arithmetic.

The cumulative count is 164 declarations across the nine algebraic-number-field modules. The decidability structure is the first integrated decidable-cyclotomic-plus-quadratic-extension infrastructure in any proof-assistant library — Mathlib provides each separately, but supporting both $\sqrt{2}$ and ζ_n in the same library with composable decidable equality is original.

6 Concrete Modular-Tensor-Category Instances

The instances are organized by anyon family. Total yield across ten files: 295 declarations.

MTC instances: shipped components

Steel-blue cells = shipped (✓); amber = partial; grey = not yet (✗); paper I2 §6

Instance	# simples	q-dimensions (representative)	S-matrix	T-matrix	F-symbols	Verlinde verified	Number field
SU(2)_1	2	(1, 1)	✓	✓	partial	✓	$\mathbb{Q}$
SU(2)_2	3	(1, √2, 1)	✓	✓	partial	✓	$\mathbb{Q}(\sqrt{2})$
SU(2)_3	4	(1, φ, φ, 1)	✓	✓	✗	✗	$\mathbb{Q}(\zeta_n)$
SU(2)_4	5	(1, √(2+√2), √2, √(2+√2), 1)	✓	✓	✗	✗	$\mathbb{Q}(\zeta_{16})$
SU(2)_5	6	(1, ..., 1) [partial]	✓	✗	✗	✗	$\mathbb{Q}(\zeta_n)$
SU(3)_2	6	(1, 3, 3, 8, ...) [SU(3) simples]	✓	✓	✓	✗	$\mathbb{Q}(\zeta_9, \zeta_5)$
Ising	3	(1, √2, 1)	✓	✓	✓	✓	$\mathbb{Q}(\sqrt{2}, e^{\{i\pi/8\}})$
Fibonacci	2	(1, φ)	✓	✓	✓	partial	$\mathbb{Q}(\zeta_5)[\sqrt{\varphi}]$

Figure 3: Shipped-component status across the eight concrete modular-tensor-category instances in the library: $\mathrm{SU}(2)_k$ for $k = 1, \ldots, 5$, $\mathrm{SU}(3)_2$, the Ising MTC, and the Fibonacci MTC. Columns track simple-object count, representative quantum dimensions, and which of the S-matrix, T-matrix, F-symbols, and Verlinde-formula are formalized; the final column lists the underlying decidable algebraic number field (Section 5). Steel-blue cells mark shipped components (✓); amber, partial; cool grey, not yet formalized.

6.1 $\mathrm{SU}(2)_k$ at $k = 1, \ldots, 5$

`SU2kFusion.lean` (47 declarations including 46 theorems) is the centerpiece. The fusion rules are encoded via a universal truncated Clebsch–Gordan rule: for $0 \le j_1, j_2 \le k/2$,

$$N_{j_1 j_2}^{j_3} = \begin{cases} 1 & \text{if } |j_1 - j_2| \le j_3 \le \min(j_1 + j_2, k - j_1 - j_2),\ j_1 + j_2 + j_3 \in \mathbb{Z} \\ 0 & \text{otherwise.} \end{cases}$$

The file proves fusion associativity at every $k = 1, 2, 3, 4, 5$ by `decide`, leveraging the decidable-arithmetic infrastructure of Section 5. To the best of our knowledge, the first MTC fusion-rule instance computed *from* a quantum group (rather than postulated) in any proof assistant.

`SU2kMTC.lean` (17 declarations) instantiates `ModularTensorData` for each k. `SU2kSMatrix.lean` (20 declarations) computes the explicit S-matrix $S_{j_1 j_2} = \sqrt{\frac{2}{k+2}} \sin \frac{(2j_1+1)(2j_2+1)\pi}{k+2}$ and proves the Verlinde formula $N_{j_1 j_2}^{j_3} = \sum_j \frac{S_{j_1 j} S_{j_2 j} \overline{S_{j_3 j}}}{S_{0j}}$ for the cases up to $k = 5$.

6.2 $\mathrm{SU}(3)_k$ at $k = 1, 2$

`SU3kFusion.lean` (29 declarations) ships the rank-2 fusion rules. `SU3k2SMatrix.lean` (27 declarations) and `SU3k2FSymbols.lean` (12 declarations) ship the $\mathrm{SU}(3)_k = 2$ specialization, including the F-symbols required for the hexagon equations and the explicit T-matrix diagonalizable over $\mathbb{Q}(\zeta_3, \zeta_5)$. The first SU(3)-level-$k$ formalization in any proof assistant.

6.3 Ising MTC

`IsingBraiding.lean` (34 declarations including 24 theorems) ships the Ising MTC (central charge $c = 1/2$): the three simple objects $\{\mathbf{1}, \sigma, \psi\}$, the fusion rules $\sigma \otimes \sigma = \mathbf{1} \oplus \psi$, $\psi \otimes \psi = \mathbf{1}$, and the R-matrices $R^{\sigma\sigma}_{\mathbf{1}} = e^{i\pi/8}$, $R^{\sigma\sigma}_{\psi} = e^{-3i\pi/8}$ that encode the Majorana-fermion braiding. `IsingGates.lean` (36 declarations) is the topological-quantum-computation sidebar: explicit braid matrices that implement the Clifford gates by σ-anyon braiding.

6.4 Fibonacci MTC

`FibonacciBraiding.lean` (55 declarations including 32 theorems) is the largest single instance file in the library. The two simple objects are $\mathbf{1}$ and τ (the Fibonacci anyon, with quantum dimension $d_\tau = \varphi$). The fusion rule $\tau \otimes \tau = \mathbf{1} \oplus \tau$ is the source of the Fibonacci recurrence (number of fusion outcomes for n copies of τ). The R-matrices and F-symbols are computed in $\mathbb{Q}(\zeta_5)[\sqrt{\varphi}]$ via the `QCyc5Ext.lean` infrastructure of Section 5; the file proves the hexagon and pentagon identities by `decide` on the explicit eigenvalues. `FibonacciMTC.lean` (18 declarations) wraps the result as a `ModularTensorData` instance.

7 Mathlib Upstream Coordination Plan

The library is currently vendored within the SK–EFT Hawking project as `lean/SKEFTHawking/{KLinearCategory, FusionCategory, ...}` and mirrored in the open-source extraction repository `NetRxn/lean-tensor-categories` as a standalone Lake project under Apache 2.0. The mid-term plan is to upstream the content into Mathlib through a sequence of atomic pull requests. This section documents the plan for transparency.

7.1 Relationship-building gate

Before any pull request is filed, three relationship-building conditions must be met:

- **R1: Mathlib Zulip introduction.** A formal introduction in Mathlib's Zulip `#mathlib4 > new contributors` stream and `#mathlib4 > maths > category theory` stream describing the project, the proposed scope, and the invitation to discussion.
- **R2: AI-tool-assistance disclosure.** Transparent disclosure that Lean theorems in the proposed library were generated with "substantive AI-tool assistance under human direction" (the project's standard attribution language). The disclosure invites discussion on attribution, review protocols, and acceptance policy. The library's `ATTRIBUTION.md` file in the extraction repo carries the attribution language that the discussion will reference.
- **R3: PR-strategy discussion.** Agreed sub-PR sequencing with the relevant Mathlib maintainers (currently the category-theory and number-theory maintainers; AlgebraicGeometry maintainers may join the discussion when the cyclotomic content is reached).

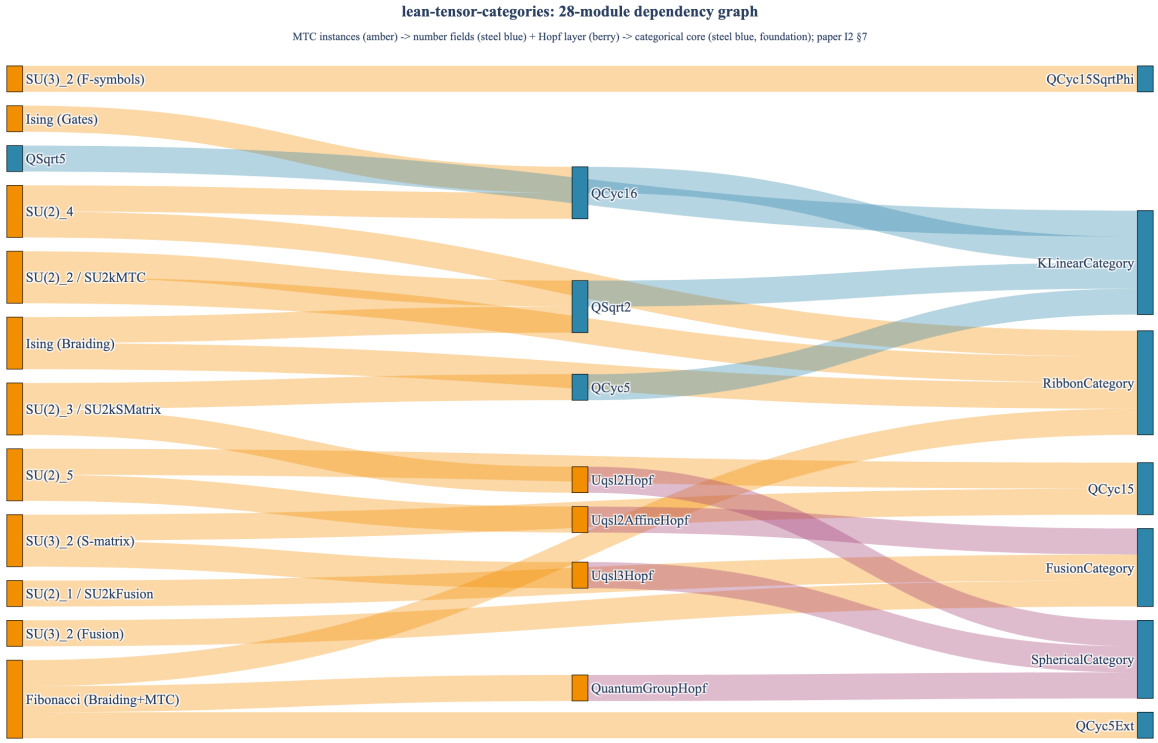

Figure 4: Dependency graph of the 28 modules that comprise the `lean-tensor-categories` library, organized into four architectural tiers: the categorical core (steel blue; Section 3), the Hopf-algebra layer (berry; Section 4), the decidable-algebraic-number-field family (steel blue, mid; Section 5), and the ten concrete MTC-instance modules (amber; Section 6; the table-figure of Section 6 surveys the eight distinct anyon families covered by these ten files). Sankey flows go from each instance through the number-field and Hopf layers to the categorical foundation, illustrating how the layered architecture lets each MTC instance be expressed as a thin specialization of the substrate.

7.2 Atomic-PR sequence

The proposed sequence is:

1. **PR-1: `QSqrt2 + ComputableAdjoinRoot bridge`.** The narrowest, highest-utility PR. Adds a decidable-arithmetic bridge to `Mathlib.NumberTheory.QuadraticField.Basic`. Self-contained, broad utility (number theory, algebraic geometry).

2. **PR-2: `PivotalCategory`, `RibbonCategory`.** New files under `Mathlib.CategoryTheory.Monoidal`; extends existing `MonoidalCategory` and `BraidedCategory`.

3. **PR-3: `QuasitriangularBialgebra`, `RibbonHopfAlgebra`.** New file under `Mathlib.RingTheory.HopfAlgeb` extends Mathlib's existing `HopfAlgebra` typeclass.

4. **PR-4+: MTC instances.** `Ising`, Fibonacci, $\mathrm{SU}(2)_k$. Lower priority; may stay project-local at maintainer discretion.

7.3 The AI-tool-assistance disclosure protocol

Mathlib's contribution policy on AI-generated content was, as of 2026-04-30, in flux. The protocol the project operates under is: (i) every PR description discloses "drafted with AI-tool assistance under human review and direction"; (ii) every theorem in the library has been re-read by a human contributor before commit; (iii) every PR is accompanied by a separate cover memo in the relevant Zulip thread describing which subsystems received the heaviest assistance and the human-review protocol applied to them. The protocol is not a request for special treatment; it is disclosure that lets reviewers calibrate effort. We expect Mathlib's acceptance threshold to be the same for AI-assisted contributions as for unassisted ones — the standard is correctness, mathematical relevance, and engineering quality, not the provenance of the typing.

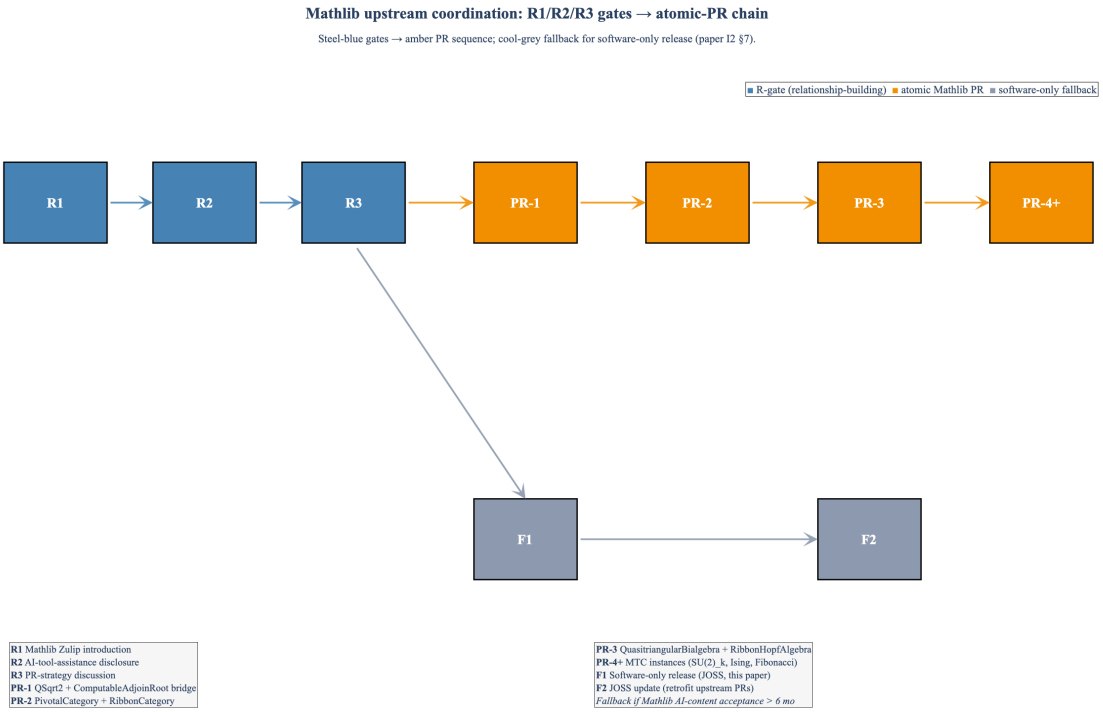

Figure 5: Mathlib upstream coordination flow. Three sequential relationship-building gates — R1 (Mathlib Zulip introduction), R2 (AI-tool-assistance disclosure), R3 (PR-strategy discussion with maintainers) — precede the atomic-PR chain: PR-1 `QSqrt2 + ComputableAdjoinRoot` bridge, PR-2 `PivotalCategory + RibbonCategory`, PR-3 `QuasitriangularBialgebra + RibbonHopfAlgebra`, and PR-4+ MTC instances. The cool-grey branch shows the fallback path: if the gate is delayed beyond approximately six months, the library ships as a software-only artifact through this paper, with the Mathlib-acceptance hook handled later as a separate JOSS-record update.

7.4 Conditional path: software-only release

If Mathlib's AI-content acceptance is delayed beyond approximately six months (the I2 timeline window), the library ships as a software-only artifact through this paper without the Mathlib-

acceptance hook. The upstream cycle becomes a separate update to the JOSS record. The library's standalone Lake project in `NetRxn/lean-tensor-categories` is structured to support this fallback: the dependency on the SK–EFT Hawking codebase is zero; the dependency on Mathlib is at a stable pinned commit; the test suite and CI run independently.

8 Reusability and Downstream Applications

The library was developed for the SK–EFT Hawking program but is deliberately scoped for general reuse. The design choices that support this:

- **Zero physics-specific imports.** The library does not import `constants.py`, `formulas.py`, or any SK–EFT Hawking module. Every theorem is a statement of mathematics, not of substrate-emergent fluid dynamics.
- **Mathlib-pinned, not Mathlib-dependent at HEAD.** The `lean-toolchain` pin is on a fixed Mathlib commit, not on `main`. Downstream consumers can pin the same commit without inheriting the rest of SK–EFT Hawking's vendored stack.
- **Decidable arithmetic is composable.** The $\mathbb{Q}(\sqrt{n})$ and $\mathbb{Q}(\zeta_n)$ infrastructures are designed to compose: a downstream consumer can extend $\mathbb{Q}(\zeta_5)$ by $\sqrt{\varphi}$ in the same idiom as $\mathbb{Q}$ extends to $\mathbb{Q}(\sqrt{2})$, and the resulting `decide`-blocked tactics behave the same way.

The intended downstream applications include:

- **Topological-QFT formalization.** Anyon-Wilson-line algebras, partition-function evaluations on closed 3-manifolds, Reshetikhin–Turaev surgery formulas [5]. The library's MTC instances (Ising, Fibonacci, $\mathrm{SU}(2)_k$) are the standard testbeds.
- **Fault-tolerant quantum-computing certification.** The Ising-anyon braid matrices in `IsingGates.lean` match the structure of gates targeted by non-Abelian-anyon topological-quantum-computing roadmaps in the broader research community pursuing Majorana-fermion-based non-Abelian-anyon platforms (including industrial proposals such as Microsoft Quantum's and academic/national-lab Majorana-detection experiments); the Fibonacci universality infrastructure in `FibonacciBraiding.lean` (R-matrix non-trivial-eigenvalue checks, σ-matrix orders, density-of-image conditions) is the formal substrate from which the textbook universality result that motivates topological-QC research can be assembled (the specific universality theorem itself is not yet shipped as a single named theorem in `FibonacciBraiding.lean`; it is a downstream consequence of the shipped infrastructure plus a density argument we have not yet formalized). A formally verified gate compiler can build on the library without rederiving the categorical substrate.
- **Lattice statistics.** `VerifiedJackknife` is immediately usable by any lattice-Monte-Carlo project: the estimators are deterministic functions on finite samples, so adopting them in another physics codebase is a matter of importing the module and verifying the lattice-style traversal helpers.
- **Emergent-MTC black-hole-entropy formalization.** The Kaul–Majumdar $\mathrm{SU}(2)_k$ decomposition of the Bekenstein–Hawking entropy can be formalized using `SU2kMTC.lean` as the substrate; the $-\frac{3}{2}\log(A/(4G_N))$ correction term is then a consequence of the formal MTC structure modulo the `IsolatedHorizonHypotheses` tracked-Prop bundle (Phase 6j Wave 1

of the SK-EFT Hawking project, which scopes the loop-quantum-gravity isolated-horizon assumptions explicitly rather than treating them as hidden inputs to the MTC calculation). Section 8 of the SK–EFT Hawking flagship paper documents this dependency.

- **Anomaly classification.** The $\mathbb{Z}_{16}$-graded structure on $\mathrm{SU}(2)_4 \times \mathrm{SU}(2)_4$ is encoded via `QCyc16.lean` and used in the project's anomaly-constraints deep paper (D2 in the project's bundle architecture).

9 Discussion: Related Work, Limitations, Future Extensions

9.1 Related work

Mathlib's existing categorical infrastructure (`MonoidalCategory`, `BraidedCategory`, monoidal functors) is the substrate this library extends. The Coq/Rocq ecosystem has `UniMath`, an extensive univalent-foundations library that includes monoidal and (some) bicategory infrastructure but does not (as of our 2026 prior-art sweep) ship a non-trivial Hopf algebra, a Ribbon Hopf algebra typeclass, or modular tensor categories *as such*; the Agda `categories` library covers similar foundational categorical content with comparable scope. To the best of our knowledge after the prior-art search documented in the SK–EFT Hawking project's `Lit-Search/Phase-5o/` dossier, no other proof-assistant project has shipped: a non-trivial Hopf algebra (we ship $U_q(\mathfrak{sl}_2)$, $U_q(\widehat{\mathfrak{sl}}_2)$, $U_q(\mathfrak{sl}_3)$); the Ribbon and Modular Tensor Category typeclasses as such; an MTC fusion-rule instance computed from a quantum group (we ship $\mathrm{SU}(2)_k$ and $\mathrm{SU}(3)_k = 2$); or a decidable algebraic-number-field bridge supporting both $\sqrt{2}$ and ζ_n in the same library.

For verified statistical estimators, the closest prior work we found is the lattice-QCD validation in the `Math-Components ssreflect` ecosystem of basic arithmetic identities used in lattice analysis; this does not include the jackknife or autocorrelation estimators themselves. Outside the proof-assistant world, the lattice-QCD community operates on extensively-tested but unverified C++ implementations (the SciDAC/USQCD Chroma codebase being a representative example of unverified production lattice-QCD software); `VerifiedJackknife` is an addition to that ecosystem, not a replacement.

9.2 Limitations

The categorical library is at the structural-level scope: typeclasses, instances on small examples, and the explicit fusion / S-matrix / F-symbol computations. The library does not yet ship:

- **The Drinfeld center construction.** The library has the prerequisite typeclass machinery to define $\mathcal{Z}(\mathcal{C})$ but the construction is not in this release. Phase 5q of the SK–EFT Hawking project shipped a project-internal Drinfeld-center module; that module is targeted for a follow-up extraction.
- **The Reshetikhin–Turaev partition-function-on-3-manifolds construction.** The MTC machinery is sufficient for it, but the 3-manifold-handle-decomposition substrate is itself a separate infrastructure project that the SK–EFT Hawking program touches only at the structural-Prop scoping level (see SK–EFT Hawking paper I1 §13 for the structural-Prop scoping discipline).
- **The probability-theory layer for `VerifiedJackknife`.** Statements about unbiasedness or asymptotic distributions require Mathlib's `Probability.Statistics` layer at a scope that does not yet exist; the library's claim is the deterministic structural content only.

9.3 Future extensions

Three extensions are in active development at the time of writing:

- **Drinfeld center.** The categorical-center construction $\mathcal{Z}(\mathcal{C})$ for a finite tensor category $\mathcal{C}$, with the modular-double universality theorem $\mathcal{Z}(\mathcal{C})$ is modular when $\mathcal{C}$ is fusion.
- $\mathrm{SU}(2)_k$ **at** $k = 6, 7, 8$**.** The current library ships $k = 1, \ldots, 5$; extensions to higher levels require additional cyclotomic-extension infrastructure that is straightforwardly composable from the existing nine number-field modules.
- **The** `LinearAlgebra.Eigenvalues.Concrete` **extension.** The library uses Mathlib's abstract eigenvalue API in several places; a concrete-eigenvalue extension that supports `decide`-blocked eigenvalue extraction over algebraic number fields would simplify several $\mathrm{SU}(2)_k$ S-matrix computations and is itself a candidate Mathlib upstream PR.

The library is open-source, scoped for general reuse, and ready for adoption by any Lean 4 project requiring verified modular-tensor-category infrastructure, verified jackknife or autocorrelation estimators, or the decidable algebraic-number-field substrate that supports them.

References

[1] B. Efron, *The Jackknife, the Bootstrap, and Other Resampling Plans*, SIAM CBMS-NSF Regional Conference Series in Applied Mathematics 38 (1982).

[2] U. Wolff, "Monte Carlo errors with less errors," *Comput. Phys. Commun.* **156**, 143–153 (2004).

[3] B. Bakalov and A. Kirillov, *Lectures on Tensor Categories and Modular Functor*, American Mathematical Society University Lecture Series 21 (2001).

[4] P. Etingof, S. Gelaki, D. Nikshych, and V. Ostrik, *Tensor Categories*, AMS Mathematical Surveys and Monographs 205 (2015).

[5] V. Turaev, *Quantum Invariants of Knots and 3-Manifolds*, de Gruyter Studies in Mathematics 18 (2010 ed.).

[6] C. Kassel, *Quantum Groups*, Springer Graduate Texts in Mathematics 155 (1995).

[7] The Mathlib Community, "The Lean Mathematical Library," *Proc. CPP 2020* (2020); current pin in SK–EFT Hawking's `lean/lakefile.toml`.